An LLM Tree-Search Harness for Triton GPU Kernel Optimization: Two Generation Engines on a 17-Operator Suite

Final Project — Parallel Programming, Spring 2026

Wei-Ke Chu
R13922198
Computer Science and Information Engineering
National Taiwan University
abat52875@gmail.com

Ju-Hsuan Wang
R13945026
Biomedical Electronics and Bioinformatics
National Taiwan University

Wen-An Wen
R13944053
Computer Science and Information Engineering
National Taiwan University

***Abstract*—We study whether large language models (LLMs) can optimize parallel GPU kernels, and which *kind* of LLM driver suits which kind of problem. We build a closed-loop harness in which an LLM proposes Triton kernels, a fixed evaluator compiles, checks, and times each candidate in an isolated subprocess, and a beam search expands the best correct candidates. Two generation engines share the one evaluator: an automated API loop driven by DEEPSEEK-V4-FLASH, and an agentic loop in which CLAUDE CODE writes and repairs kernels over several reasoning steps. We deliberately include hard kernels the cheap loop cannot solve and design a handoff contract (`HARNESS.md`) that routes them to the agent, specifying what it reads, writes, and its iteration budget. On a 17-operator suite benchmarked against PyTorch eager on an NVIDIA RTX 5090, the API loop reaches $4.6\times$ on KL-divergence and 2.2–$4.3\times$ on RoPE, Welford, RMSNorm, and cross-entropy, while honestly reporting $\approx 1.0\times$ on five operators already at the memory roofline. The same engine fails completely on the hardest compute-bound problem, 4-bit weight GEMM: 0 of 12 candidates compiled to a correct kernel, and scaling the generator to the larger DEEPSEEK-V4-PRO does not fix it (still 0/12). The agentic engine solved it to $1.09\times$ over a cuBLAS-plus-dequantize baseline, and reached about $5\times$ on fused-linear cross-entropy, another compute-bound problem the API loop could not crack. The engines are complementary. Every number is derived mechanically from per-candidate run logs at a total LLM cost of $0.22. Code, run logs, and the scripts that regenerate every figure and table are available at https://github.com/abattada/Tree-Harness-Kernel-Optimization.**

***Index Terms*—GPU kernels, Triton, LLM code generation, AI agents, performance optimization, roofline**

Project category. This is a **Category B** project (AI-agent code optimization): we optimize parallel GPU kernels with two LLM-driven engines and analyze, in particular, the cases where the agents fail.

I. INTRODUCTION

Writing fast GPU kernels is expensive expert labor. A kernel's performance hinges on tiling, vectorization, memory-access patterns, and launch parameters (block sizes, warps, pipeline stages), and the interaction of these choices is hard to predict. We ask whether large language models can do this optimization, and—the sharper question—*which style of LLM driver fits which kind of kernel*. We compare a cheap, high-throughput API loop against an agentic loop that can reason across steps and repair its own errors.

We built a closed-loop harness that treats kernel optimization as an LLM-in-the-loop tree search over Triton [1] implementations. Candidate code never touches the clock: all timing and correctness checking live in a fixed evaluator, so reported speedups are defensible. Two generation engines plug into that one evaluator. A central design choice is that the suite *deliberately* includes hard, compute-bound kernels that the cheap one-shot loop cannot solve, together with a contract (`HARNESS.md`) that hands those operators to an agent able to iterate. Our contributions are (i) a reproducible harness with a strict provenance discipline; (ii) *a designed agent-handoff contract* (Fig. 2) that routes the operators the API loop fails on to an agentic LLM, specifying exactly what the agent reads, what it writes, and its iteration budget, so both engines feed one comparison pipeline; and (iii) an honest result—the API loop, even scaled to a larger model, cannot solve the hard compute-bound kernels (`int4_gemm`, `flce`) that the agent solves, to near hardware peak and $\sim 5\times$ respectively.

II. BACKGROUND AND RELATED WORK

Benchmark and baseline. Our suite has 17 operators across elementwise, reduction, normalization, loss, matmul, and positional categories, chosen to mirror the Liger operator family [2] used in LLM training. The baseline for every operator is PyTorch *eager* execution of a reference implementation; the kernel's job is to match the reference within tolerance and run faster. Twelve operators have optimization headroom; five (`vector_add`, `vector_exp`, `sum`, `softmax`, `layer_norm`) already saturate memory bandwidth and serve as a control group. We use the full suite, not a subset.

LLMs for kernels. KernelBench [3] asks whether LLMs can write efficient GPU kernels and reports that correctness, not just speed, is the binding constraint. Production libraries such as Liger [2] show that a small set of fused Triton kernels removes most intermediate-tensor traffic in training; the largest wins come from the same idea we exploit—fusing multi-pass references into a single pass.

Agent setup. Engine A calls DEEPSEEK-V4-FLASH (thinking enabled) through an OpenAI-compatible API. Engine B is CLAUDE CODE (model CLAUDE OPUS, reasoning effort "extra high") operating under a written contract (`HARNESS.md`) that lets it write a kernel, invoke the evaluator, read the traceback or measurement, and iterate. The verbatim prompts and configuration are in the Appendix.

III. METHODOLOGY

A. Closed loop and the two engines

Fig. 1 shows the harness. It seeds an initial population from design hints, asks the LLM to emit a Triton module exposing `triton_run(*inputs)`, evaluates each candidate, scores the correct ones against a roofline anchor, and expands a top-k beam into the next round. A knowledge base (KB) of previously winning kernels is retrieved as few-shot exemplars and rebuilt from the logs between batches.

B. Prompt-engineering design

The *system prompt* fixes the role, the target hardware, the hard constraints (define only `triton_run`; no direct call to the reference op; allowed imports; Triton 3.6 API pitfalls), and the required output format (one Python block plus a confidence line). It is identical across candidates so that DeepSeek's prefix cache is hit. The *input prompt* is built per candidate: operator signature, tolerance, the PyTorch reference source, the assigned optimization strategy, the parent kernel and its measurement when refining, and KB exemplars. Both are reproduced verbatim in Appendices A and B.

C. Iteration strategy and two-layer search

The outer loop explores *algorithmic structure* (fusion, single-pass reductions, persistent kernels, split-K). Within a candidate, tunable launch and tile parameters are exposed as an LLM-chosen grid and swept by Triton's autotuner; the evaluator records the winning configuration so the next round can narrow the grid. A pilot showed that most of the gain comes from a good seed plus a short error-fix loop, so we foreground a minimal tree with free repair and keep the strategy/bandit/scorer mechanisms as ablation toggles.

D. The agent-handoff contract (`HARNESS.md`)

The operators the API loop cannot solve are not left unsolved: we designed a written contract that hands them to an agent (Fig. 2). The contract fixes three things. *What the agent reads*: a task card with the operator signature, tolerance, the PyTorch reference, forbidden patterns, and the roofline target, plus KB exemplars and, for a resumed task, the handoff state (`STATE.md`, `best.py`, and the journal tail). *What the agent writes*: one candidate file per attempt, an appended journal row through the same `eval_one` evaluator the API loop uses (so the results are directly comparable), a `STATE.md` carrying status, what was tried, pitfalls, and next steps for the following session, and a `best.py` exported from the journal. *Its iteration scope*: the budget is simply the journal line count, and the agent loops write $\rightarrow$ evaluate $\rightarrow$ read (a traceback or a measurement) $\rightarrow$ refine until the budget is spent or the kernel saturates the roofline. Because the agent calls the identical evaluator, its kernels are timed and checked exactly like the API loop's, and both feed one comparison.

TABLE I
DEEPSEEK API-LOOP TOTALS (14 OPERATORS).

Metric	Value	Unit
Candidates generated	222	kernels
Passed correctness	74	% of candidates
Operators with $>1\times$ speedup	71	% of operators
Operators with $>1.2\times$ speedup	43	% of operators
GPU evaluation time	15.4	minutes
LLM tokens used	1.52	million

IV. EXPERIMENTAL SETUP

All measurements come from one evaluator run as an isolated subprocess bound to a single GPU via `CUDA_VISIBLE_DEVICES`, so exactly one candidate is timed per card at a time. Both the candidate and the PyTorch reference are timed with `triton.testing.do_bench` (median over 100 reps after 25 warmups), and both include output allocation, removing an asymmetry that would flatter the kernel. Correctness uses `torch.allclose` at per-operator tolerances against the eager reference with a fixed input seed. Hardware is an NVIDIA RTX 5090 (Blackwell, sm_120, 32 GB GDDR7, $\sim$1790 GB/s peak); software is PyTorch (CUDA 12.8) with Triton 3.6. A static anti-cheat check rejects candidates that call the reference op directly (tagged `cheat`). We define **success** as a candidate that is correct within tolerance and faster than eager (speedup > 1); a **failure** is any incorrect, non-compiling, timed-out, or cheating candidate. Every run writes a per-candidate `journal.jsonl` (the single source of truth: correctness, speedup, bandwidth, winning configuration, token/GPU cost, full code) plus round metrics and full LLM transcripts; all tables and figures here are generated from those logs with no hand-entered numbers.

V. RESULTS

On the memory-bound suite the API loop delivers large, defensible speedups (Fig. 3). The operators whose references waste the most memory traffic win biggest: KL-divergence at $4.61\times$ and RoPE at $4.29\times$, followed by Welford ($2.43\times$), RMSNorm ($2.32\times$), and cross-entropy ($2.18\times$), each by fusing a multi-pass reference into one pass. The five control operators land at 0.99–$1.07\times$: the engine correctly finds no headroom where the reference already saturates bandwidth. Across 222 candidates on 14 operators, 74% were correct and 43% of operators cleared $1.2\times$ (Table I).

The agentic engine was pointed at `int4_gemm`: a 4-bit weight, fp16 activation matrix multiply that must dequantize packed weights inside the kernel, beating a baseline that dequantizes to a full fp16 matrix and calls cuBLAS. Its first two attempts failed; the third was correct at $0.99\times$, and it then climbed a tuning ladder to $1.09\times$ (Fig. 4). The decisive moves were structural—dropping a 3-D reshape and loading weights

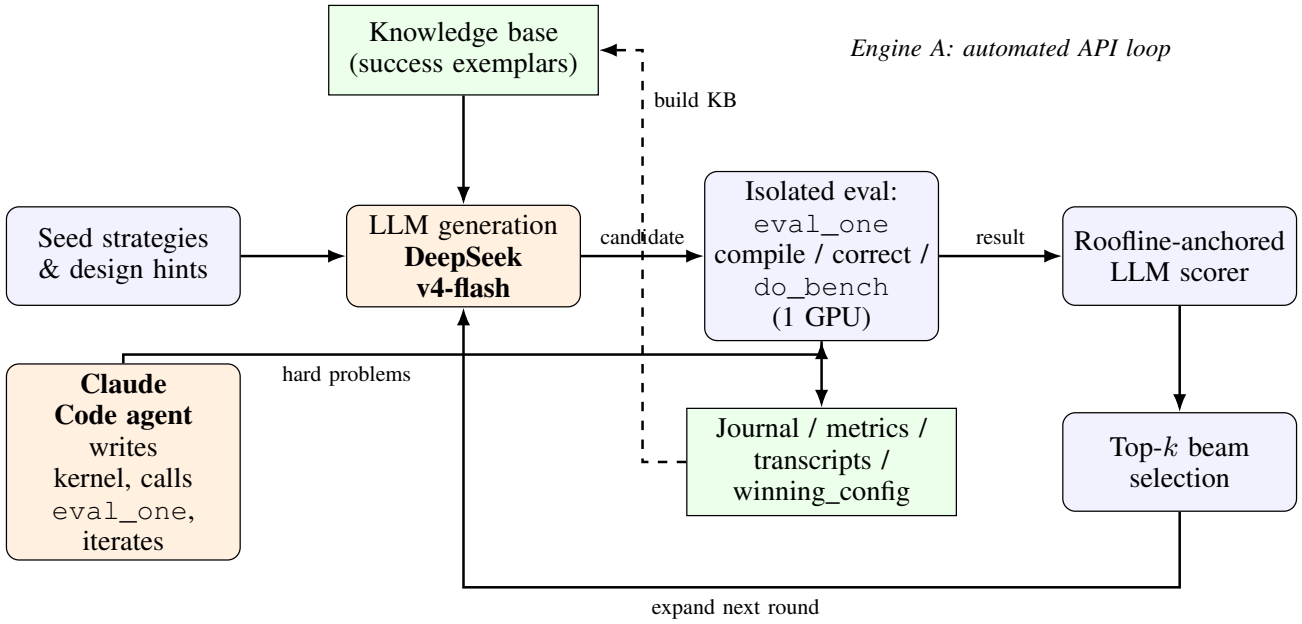

Fig. 1. The closed-loop harness. Engine A (DeepSeek API loop) and Engine B (Claude Code agent) are two entry points into one shared evaluator; all persisted artifacts (logs, transcripts, winning configurations) are produced by the evaluator, not the generator.

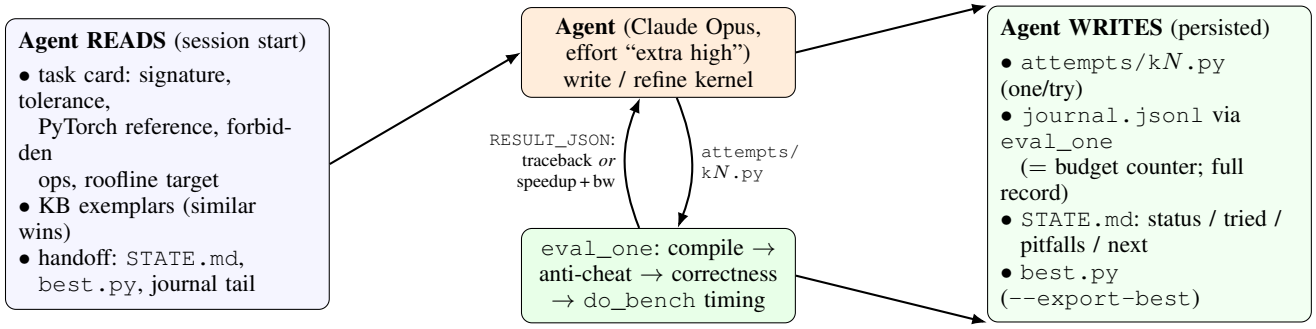

Fig. 2. The HARNESS.md agent-handoff contract: what the agent reads at the start of a session, what it writes (all through the same eval_one evaluator as Engine A), and its iteration scope. This is how operators that the one-shot API loop cannot solve are completed by an agent.

as a [BLOCK_K,BLOCK_N] tile via redundant, L2-cached rows—and a launch sweep settling on a $128 \times 64 \times 64$ tile, GROUP_M=4, num_warps=8, num_stages=2. At $1.09\times$ the kernel sits at $\approx 89\%$ of fp16 tensor-core peak; the agent declared saturation by a hardware-limit argument rather than spending more budget.

The engines diverge sharply on the hardest, compute-bound operators (Table II). On int4_gemm the API loop produced 12 candidates and none were correct; the agent reached $1.09\times$. The pattern repeats on the other hard operators. On fused-linear cross-entropy (flce) the flash API loop produced 0 of 12 correct and pro managed only $1.21\times$, whereas the agent reached about $5\times$ (best $5.1\times$, independently re-measured at $4.6\times$ on an idle GPU) by fusing the logit reduction so the 268 MB logit tensor is never materialized. On plain gemm no engine beats cuBLAS: the agent's best ($1.01\times$, 9/14 correct) merely matches it, as expected for an already-optimal fp16 GEMM. The hard kernels reward the agent's compile–read–repair loop.

A. Scaling the generator: flash vs. pro

To test whether the API loop's failures are simply a matter of a weak model, we re-ran it with the larger DEEPSEEK-V4-PRO at two budgets, 12 and 24 candidates per operator. On memory-bound operators flash and pro are tied, and the extra budget barely moves the result (Table III): KL-divergence stays at $4.61\times$, cross-entropy at $2.18\times$, and the rest within a few percent. The only visible budget effect

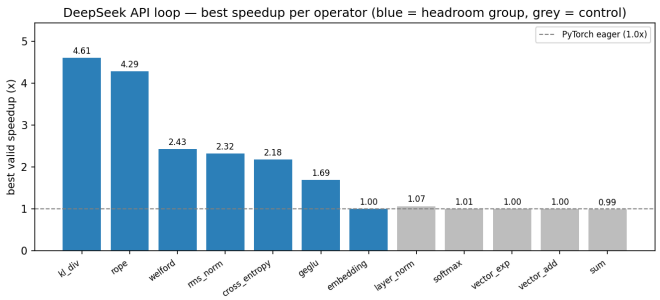

Fig. 3. DeepSeek API loop: best valid speedup per operator over PyTorch eager. Blue: headroom group; grey: control group at the memory roofline.

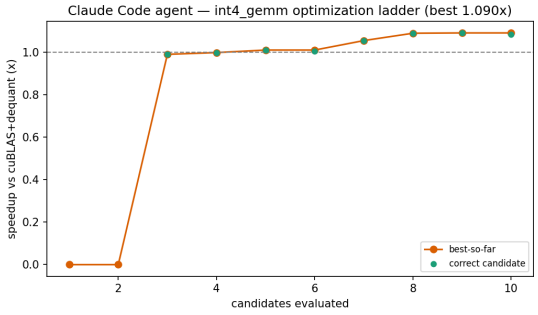

Fig. 4. Claude Code agent on `int4_gemm`: best-so-far speedup over a cuBLAS+dequantize baseline. Two failed attempts, then a correct kernel and a tuning ladder to $1.09\times$.

is `welford`, where pro rises from 2.17 to $2.42\times$ as more attempts lift its correctness rate. The ceiling here is the shared roofline, not the model or the budget. (Pro (24) was re-timed on an exclusive GPU; an earlier shared-GPU run had contention-inflated references and is discarded—timing requires an exclusive card.) The larger model does help on one hard case—it cracked `flce` (2/8 correct, $1.21\times$) where flash got 0/12—but `int4_gemm` remained 0/12 for *both* models, and `gemm`/`addmm` stayed below the cuBLAS baseline. Scaling the single-shot generator does not close the gap that the agent closes (the agent reaches $5\times$ on `flce` and $1.09\times$ on `int4_gemm`).

VI. Discussion and Analysis

The central finding is complementarity, and the failures explain why. We did a root-cause analysis of the cases where Engine A failed.

Why `int4_gemm` failed for the API loop (0/12). The errors split across three causes recorded in the logs. *Runtime* errors dominated: the kernel must unpack eight 4-bit weights from each 32-bit word with bit shifts and masks, align them to the K-loop, and subtract the zero point—an indexing problem the single-shot model got wrong (out-of-bounds and dtype mismatches). One candidate produced *wrong output* (a correct-looking but mis-scaled dequantization), and several were rejected as *cheat* for falling back to a high-level matmul. The task needs several interdependent steps to be correct

TABLE II
HEAD-TO-HEAD ON THE HARDEST OPERATORS (CORRECT/ATTEMPTS; BEST SPEEDUP WHERE ANY CANDIDATE IS CORRECT).

Operator	DS-flash	DS-pro	Claude (B)	Best
`int4_gemm`	0/12	0/12	8/10 @1.09	**1.09**× (B)
`flce`	0/12	2/8 @1.21	6/7 @5.1	**5.1**× (B)
`gemm`	0/12	3/12 @0.94	9/14 @1.01	1.01× (B)

TABLE III
MEMORY-BOUND OPERATORS: BEST SPEEDUP FOR DEEPSEEK-FLASH AND -PRO AT CANDIDATE BUDGETS OF 12 AND 24. NUMBERS IN PARENTHESES ARE THE COLUMN HEADER BUDGETS; PRO (24) WAS RE-TIMED ON AN EXCLUSIVE GPU. FLASH AND PRO ARE TIED, AND RAISING THE PRO BUDGET FROM 12 TO 24 CHANGES LITTLE.

Operator	flash (24)	pro (12)	pro (24)
`rope`	4.29×	4.41×	3.96×
`kl_div`	4.61×	4.60×	4.61×
`rms_norm`	2.32×	2.30×	2.30×
`cross_entropy`	2.18×	2.11×	2.18×
`welford`	2.43×	2.17×	2.42×
`geglu`	1.69×	1.68×	1.68×

simultaneously; a generator with no feedback rarely lands all of them at once. The agent succeeded precisely because it could compile, read the traceback, and fix one cause at a time.

Why fused-linear cross-entropy failed for the API loop, and how the agent won. For flash all 12 failures were runtime errors; pro got it correct twice but only to $1.21\times$. The operator fuses a matmul with a logsumexp and a gathered negative log-likelihood without materializing the full logits; getting the tiled online reduction and the target gather correct in one shot is beyond the single-pass generator. The agent, iterating, reached about $5\times$—the largest single win in the suite—because the fused kernel never writes the 268 MB logit tensor that the eager reference materializes. This is the clearest case for the agentic engine: the same problem that yields 0/12 to one-shot generation is a $5\times$ win once the loop can repair and refine.

Is it just a weak model? No. Replacing flash with the larger pro model left memory-bound speedups unchanged (the roofline, not the model, is the limit there) and still produced 0/12 correct on `int4_gemm`. The bigger model did crack `flce` ($1.21\times$), so model capacity helps at the margin on hard problems—but the decisive factor for the hardest kernel was the agent's ability to compile, read the error, and fix one cause at a time, not raw model strength.

Where the API loop wins. Memory-bound operators reward a single good idea—fuse the passes—that the model already knows from the Liger family; correctness is then easy and the speedup follows. Budget curves (not shown) place most of the gain in the first few candidates, which argues for spending budget on seed diversity and fast repair rather than deep refinement. The control group matters: reporting $\approx 1.0\times$ where the roofline leaves no room is what makes the large speedups credible.

Threats to validity. Speedups are measured on one GPU and one input shape per operator; the roofline uses a vendor

peak rather than a measured stream ceiling; and the agent result covers a single operator.

VII. Conclusion

An LLM-in-the-loop tree search over Triton kernels yields real, verified speedups on memory-bound operators (2–4.6×) at a total cost near $1. The choice of driver matters: a cheap API loop is the right tool for fuse-the-passes problems, while a hard compute-bound kernel like 4-bit GEMM needs an agent that can reason across steps and repair its own errors. The two engines are complementary, and the failure analysis—not just the speedups—is the useful product.

Acknowledgment

Kernels were generated by DeepSeek-V4-Flash and Claude Code as the object of study; reported numbers trace to the run logs in the project repository.

References

[1] P. Tillet, H. T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.

[2] P.-L. Hsu, Y. Dai, V. Kothapalli, Q. Song, S. Tang, S. Zhu *et al.*, "Liger kernel: Efficient triton kernels for llm training," *arXiv preprint arXiv:2410.10989*, 2024.

[3] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "Kernelbench: Can llms write efficient gpu kernels?" *arXiv preprint arXiv:2502.10517*, 2025.

Appendix A
Input Prompt

Verbatim per-candidate input prompt for the RoPE operator (the run that produced the 4.29× result in Fig. 3). For Engine B (Claude Code, model Claude Opus, reasoning effort "extra high") the user prompt is the one-line task "Read HARNESS.md, optimize int4_gemm, budget 28 evals, use GPU N," with `HARNESS.md` as the standing system contract.

````
OPERATOR: rope  (category: misc)
SIGNATURE: triton_run(x: f16[8, 32, 4096, 128], cos
    : f16[4096, 64], sin: f16[4096, 64]) -> f16[8,
    32, 4096, 128]  # rotate-half RoPE: out = cat(
    x1*cos - x2*sin, x1*sin + x2*cos) where x1,x2 =
     split(x, 64, dim=-1)
TOLERANCE: rtol=0.02, atol=0.02
PYTORCH REFERENCE:
```python
def _rope_ref(x, cos, sin):
    d2 = x.shape[-1] // 2
    x1, x2 = x[..., :d2], x[..., d2:]
    c = cos.view(1, 1, *cos.shape)
    s = sin.view(1, 1, *sin.shape)
    return torch.cat([x1 * c - x2 * s, x1 * s + x2
    * c], dim=-1)
```
ASSIGNED STRATEGY: free refinement --- analyze the
    feedback below and make the single most
    promising improvement.

PARENT VERSION (round 0, strategy=seed):
```python
import torch
import triton
import triton.language as tl


@triton.jit
def rope_kernel(
    x_ptr, cos_ptr, sin_ptr, out_ptr,
    stride_x_b, stride_x_h, stride_x_s, stride_x_d,
    stride_cos_s, stride_cos_d,
    stride_sin_s, stride_sin_d,
    stride_out_b, stride_out_h, stride_out_s,
    stride_out_d,
    B: tl.constexpr, H: tl.constexpr, S: tl.
    constexpr, D: tl.constexpr,
    HALF_D: tl.constexpr,
    ROWS_PER_PROG: tl.constexpr,
):
    pid = tl.program_id(0)
    start_row = pid * ROWS_PER_PROG

    # Offsets for the half dimension (0..63),
    assuming HALF_D=64
    offs_half = tl.arange(0, HALF_D)

    for i in range(ROWS_PER_PROG):
        row = start_row + i
        # Decompose linear row index into (b, h, s)
        b = row // (H * S)
        rem = row % (H * S)
        h = rem // S
        s = rem % S

        # Compute base pointers for this row
        x_base = x_ptr + b * stride_x_b + h *
    stride_x_h + s * stride_x_s
        out_base = out_ptr + b * stride_out_b + h *
     stride_out_h + s * stride_out_s
        cos_base = cos_ptr + s * stride_cos_s
        sin_base = sin_ptr + s * stride_sin_s

        # Load x halves (contiguous along last
    dimension)
        x1 = tl.load(x_base + offs_half *
    stride_x_d, eviction_policy='evict_first')
        x2 = tl.load(x_base + (offs_half + HALF_D)
    * stride_x_d, eviction_policy='evict_first')

        # Load cos and sin for this sequence
    position
        c = tl.load(cos_base + offs_half *
    stride_cos_d, eviction_policy='evict_first')
        s_val = tl.load(sin_base + offs_half *
    stride_sin_d, eviction_policy='evict_first')

        # Compute rotated halves
        out1 = x1 * c - x2 * s_val
        out2 = x1 * s_val + x2 * c

        # Store output
        tl.store(out_base + offs_half *
    stride_out_d, out1)
        tl.store(out_base + (offs_half + HALF_D) *
    stride_out_d, out2)


def triton_run(x: torch.Tensor, cos: torch.Tensor,
    sin: torch.Tensor) -> torch.Tensor:
    assert x.dtype == torch.float16
    assert cos.dtype == torch.float16 and sin.dtype
     == torch.float16
    B, H, S, D = x.shape
    assert D == 128
    assert cos.shape == (S, D // 2) and sin.shape
    == (S, D // 2)

    out = torch.empty_like(x)

    HALF_D = D // 2       # 64
````
````

```
    ROWS_PER_PROG = 8
    total_rows = B * H * S
    assert total_rows % ROWS_PER_PROG == 0, "
    total_rows must be divisible by ROWS_PER_PROG"

    grid = (total_rows // ROWS_PER_PROG,)

    rope_kernel[grid](
        x, cos, sin, out,
        x.stride(0), x.stride(1), x.stride(2), x.
    stride(3),
        cos.stride(0), cos.stride(1),
        sin.stride(0), sin.stride(1),
        out.stride(0), out.stride(1), out.stride(2)
    , out.stride(3),
        B, H, S, D,
        HALF_D,
        ROWS_PER_PROG,
        num_warps=4,
        num_stages=2,
    )
    return out
```
PARENT MEASUREMENT: correct=True speedup=3.939x
    triton=0.4075ms vs pytorch=1.6054ms
    achieved_bw=1317GB/s (74% of peak)

ALREADY TRIED ON THIS BRANCH: seed→3.94x
Avoid repeating a failed direction unless you fix
    its root cause.

REFERENCE KERNEL --- a previously successful kernel
     for this operator, measured 3.94x at 74% peak
    bandwidth. Reuse the working idioms (API usage,
     masking, launch setup); do not copy blindly if
     shapes/semantics differ:
```python
import torch
import triton
import triton.language as tl

@triton.jit
def rope_kernel(
    x_ptr, cos_ptr, sin_ptr, out_ptr,
    stride_x_b, stride_x_h, stride_x_s, stride_x_d,
    stride_cos_s, stride_cos_d,
    stride_sin_s, stride_sin_d,
    stride_out_b, stride_out_h, stride_out_s,
    stride_out_d,
    B: tl.constexpr, H: tl.constexpr, S: tl.
    constexpr, D: tl.constexpr,
    HALF_D: tl.constexpr,
    ROWS_PER_PROG: tl.constexpr,
):
    # Each program processes ROWS_PER_PROG
    consecutive rows.
    pid = tl.program_id(0)
    start_row = pid * ROWS_PER_PROG
    offs_half = tl.arange(0, HALF_D)

    for i in range(ROWS_PER_PROG):
        row = start_row + i
        # Decompose linear row index into (b, h, s)
        b = row // (H * S)
        rem = row % (H * S)
        h = rem // S
        s = rem % S

        # Base pointers for x and output at this
    row
        x_base = x_ptr + b * stride_x_b + h *
    stride_x_h + s * stride_x_s
        out_base = out_ptr + b * stride_out_b + h *
     stride_out_h + s * stride_out_s

        # Load x halves (no mask needed because
    offsets always < D)
        x1 = tl.load(x_base + offs_half,
    eviction_policy='evict_first')
        x2 = tl.load(x_base + (offs_half + HALF_D),
     eviction_policy='evict_first')

        # Load cos and sin for this sequence
    position
        cos_base = cos_ptr + s * stride_cos_s
        sin_base = sin_ptr + s * stride_sin_s
        c = tl.load(cos_base + offs_half,
    eviction_policy='evict_first')
        s_val = tl.load(sin_base + offs_half,
    eviction_policy='evict_first')

        # Compute rotated halves
        out1 = x1 * c - x2 * s_val
        out2 = x1 * s_val + x2 * c

        # Store output
        tl.store(out_base + offs_half, out1)
        tl.store(out_base + offs_half + HALF_D,
    out2)


def triton_run(x: torch.Tensor, cos: torch.Tensor,
    sin: torch.Tensor) -> torch.Tensor:
    assert x.dtype == torch.float16
    assert cos.dtype == torch.float16 and sin.dtype
     == torch.float16
    B, H, S, D = x.shape
    assert D == 128
    assert cos.shape == (S, D // 2) and sin.shape
    == (S, D // 2)

    out = torch.empty_like(x)

    HALF_D = D // 2  # 64
    ROWS_PER_PROG = 8
    total_rows = B * H * S
    assert total_rows % ROWS_PER_PROG == 0, "grid-
    stride loop assumes exact division"

    grid = (total_rows // ROWS_PER_PROG,)

    rope_kernel[grid](
        x, cos, sin, out,
        x.stride(0), x.stride(1), x.stride(2), x.
    stride(3),
        cos.stride(0), cos.stride(1),
        sin.stride(0), sin.stride(1),
        out.stride(0), out.stride(1), out.stride(2)
    , out.stride(3),
        B, H, S, D,
        HALF_D,
        ROWS_PER_PROG,
        num_warps=4,
        num_stages=2,
    )
```

REFERENCE KERNEL --- a previously successful kernel
     for this operator, measured 3.35x at 63% peak
    bandwidth. Reuse the working idioms (API usage,
     masking, launch setup); do not copy blindly if
     shapes/semantics differ:
```python
import torch
import triton
import triton.language as tl
```

```
@triton.jit
def rope_kernel(
    x_ptr, cos_ptr, sin_ptr, out_ptr,
    stride_x_b, stride_x_h, stride_x_s, stride_x_d,
    stride_cos_s, stride_cos_d,
    stride_sin_s, stride_sin_d,
    stride_out_b, stride_out_h, stride_out_s,
    stride_out_d,
    B: tl.constexpr, H: tl.constexpr, S: tl.
    constexpr, D: tl.constexpr,
    HALF_D: tl.constexpr,
    BLOCK_HALF: tl.constexpr,
):
    pid = tl.program_id(0)
    total_rows = B * H * S
    if pid >= total_rows:
        return

    # Decompose pid to (b, h, s)
    b = pid // (H * S)
    rem = pid % (H * S)
    h = rem // S
    s = rem % S

    # Base offsets for this row
    base_x = b * stride_x_b + h * stride_x_h + s *
    stride_x_s
    base_out = b * stride_out_b + h * stride_out_h
    + s * stride_out_s

    # Offsets for half dimension (0..63)
    offs_half = tl.arange(0, BLOCK_HALF)

    # Load first half x1 (contiguous)
    x1 = tl.load(
        x_ptr + base_x + offs_half * stride_x_d,
        eviction_policy='evict_first',
    )

    # Load second half x2 (contiguous, starts at
    offset 64)
    x2 = tl.load(
        x_ptr + base_x + (offs_half + HALF_D) *
    stride_x_d,
        eviction_policy='evict_first',
    )

    # Load cos and sin for this position (evict
    first since used once)
    cos_vals = tl.load(
        cos_ptr + s * stride_cos_s + offs_half *
    stride_cos_d,
        eviction_policy='evict_first',
    )
    sin_vals = tl.load(
        sin_ptr + s * stride_sin_s + offs_half *
    stride_sin_d,
        eviction_policy='evict_first',
    )

    # Compute rotated halves
    out1 = x1 * cos_vals - x2 * sin_vals
    out2 = x1 * sin_vals + x2 * cos_vals

    # Store results
    tl.store(
        out_ptr + base_out + offs_half *
    stride_out_d,
        out1,
    )
    tl.store(
        out_ptr + base_out + (offs_half + HALF_D) *
     stride_out_d,
        out2,
    )


def triton_run(x: torch.Tensor, cos: torch.Tensor,
    sin: torch.Tensor) -> torch.Tensor:
    assert x.dtype == torch.float16
    assert cos.dtype == torch.float16 and sin.dtype
     == torch.float16
    B, H, S, D = x.shape
    assert D == 128
    assert cos.shape == (S, D // 2) and sin.shape
    == (S, D // 2)

    out = torch.empty_like(x)
```

This is round 1. Produce the module now, following the OUTPUT FORMAT exactly.

APPENDIX B
SYSTEM PROMPT

Verbatim system prompt (identical across all Engine A candidates; the cached prefix).

```
You are an expert GPU kernel engineer writing
    Triton kernels.

TARGET HARDWARE: NVIDIA RTX 5090 (Blackwell, sm_120
    ), 32GB GDDR7, ~1790 GB/s peak bandwidth

TASK: given a PyTorch reference operator, produce a
     complete Python module that
implements it with a Triton kernel, optimized for
    the target hardware.

HARD CONTRACT (violations are auto-rejected):
1. The module MUST define `def triton_run(*inputs)
    -> torch.Tensor` with the
   exact signature documented per operator. It
    allocates the output, launches
   the kernel(s), and returns the output tensor.
2. The core computation MUST be done in Triton
    kernels you write. Calling the
   equivalent PyTorch op (e.g. torch.softmax for
    the softmax task) is cheating
   and is rejected by a static checker. Using torch
     only for output allocation
   (torch.empty/empty_like) and trivial glue is
    fine.
3. No benchmarking, printing, or timing code in the
     module. The harness times
   `triton_run` externally with triton.testing.
    do_bench.
4. The module must be self-contained: only `import
    torch`, `import triton`,
   `import triton.language as tl`, `math`, and
   `from triton.language.extra import libdevice`.
    Top-level code runs at
   import time --- keep it to definitions only.
5. Numerical tolerance per operator is given in the
     task. Stay within it.

OUTPUT FORMAT (exactly):
- One fenced python code block containing the full
    module.
- After the code block, a single line:
  CONF: {"confidence": <0-100 integer>, "notes": "<
    one sentence>"}
  where confidence is your honest estimate that the
     module compiles AND passes
  the correctness check on the first try.
```

```
STRATEGY LIBRARY (id | kind | applies | description
    ):
- tune_block_size | launch | elementwise,loss,
    matmul,misc,normalization,reduction | Sweep
    BLOCK sizes (and 2D tile shapes). Pick the
    value that balances occupancy vs. register
    pressure; powers of two from 128 to 8192 for 1D
    .
- tune_warps_stages | launch | elementwise,loss,
    matmul,misc,normalization,reduction | Tune
    num_warps (1..16) and num_stages (2..6) for the
     chosen block size. Small rows want fewer warps
    ; pipelined loads want more stages.
- grid_loop_remap | launch | elementwise,misc,
    reduction | Launch fewer programs, each looping
     over multiple tiles (grid-stride loop) to
    amortize launch and scheduling overhead.
- constexpr_specialize | launch | elementwise,loss,
    matmul,misc,normalization,reduction |
    Specialize shapes as tl.constexpr and use tl.
    static_assert / tl.multiple_of / tl.
    max_contiguous hints so the compiler can
    vectorize and drop boundary masks.
- vectorized_access | memory | elementwise,loss,
    matmul,misc,normalization,reduction | Make
    every load/store wide and coalesced: contiguous
     tl.arange ranges, process several elements per
     thread, align block boundaries to 128B.
- multirow_per_program | memory | loss,
    normalization,reduction | Process multiple rows
     per program (2D block) so short rows stop
    underutilizing the SM; tune ROWS_PER_PROG.
- cache_eviction_hints | memory | elementwise,loss,
    matmul,misc,normalization,reduction | Use cache
     modifiers/eviction policies (e.g.
    eviction_policy='evict_first' for streaming
    inputs, 'evict_last' for reused tiles).
- mask_elimination | memory | elementwise,misc,
    normalization | When sizes divide the block
    evenly, drop masks entirely (or split the grid
    into a full-tile fast path + boundary path).
- one_pass_welford | structure | normalization,
    reduction | Replace two-pass mean/variance with
     a single-pass Welford or sum/sumsq computation
     to halve memory traffic.
- online_softmax | structure | loss,normalization |
     Use the online (single-pass) softmax/logsumexp
     recurrence to fuse the max and sum passes and
    reduce reads of the row.
- persistent_kernel | structure | loss,matmul,
    reduction | Use a persistent-kernel formulation
    : #programs = #SMs, each program loops over
    work items; combine partial results with one
    final pass (avoid atomics where possible).
- two_stage_reduction | structure | loss,reduction
    | Tune the two-stage reduction split: partial-
    sum count and per-stage block sizes. Stage 2
    must also be a Triton kernel --- calling torch
    reduction APIs (.sum()/torch.sum) for the final
     combine is rejected.
- kernel_fusion | structure | elementwise,loss,misc
    ,normalization | Fuse adjacent computations
    into one kernel so intermediates stay in
    registers (e.g. matmul tile + logsumexp + NLL
    for fused CE).
- tiling_shapes | structure | matmul | Tune BLOCK_M
    /BLOCK_N/BLOCK_K for tensor-core shapes (
    multiples of 16); try wide-N vs square tiles;
    ensure K-loop is pipelined.
- group_swizzle | structure | matmul | Reorder
    program ids in grouped/swizzled order (GROUP_M)
     to improve L2 reuse across neighboring tiles.
- split_k | structure | matmul | Use split-K (
    parallelize the K dimension across programs,
    then reduce) when M*N is small relative to K or
     occupancy is low.
- fast_math | numeric | elementwise,loss,
    normalization | Use faster math: exp2/log2 with
     rescaling instead of exp/log, rsqrt instead of
     1/sqrt, fdiv fast paths --- keep within
    tolerance.
- precision_strategy | numeric | matmul | Tune tl.
    dot input_precision / accumulator dtype (fp16
    in, fp32 acc) and output converts; ensure
    tensor cores are engaged.
- recompute_vs_store | numeric | loss,normalization
    ,reduction | Trade recomputation for memory
    traffic: keep row statistics in registers and
    recompute cheap values instead of re-reading
    DRAM.
- atomic_free_combine | structure | loss,reduction
    | Avoid global atomics: write per-program
    partials and combine in a second tiny kernel or
     with a deterministic tree reduction.

GUIDANCE:
- You will be assigned ONE strategy per attempt.
    Apply that strategy as the
  primary change; keep other aspects close to the
    parent version unless the
  strategy requires restructuring.
- Prefer correctness over aggressiveness: a fast-
    but-wrong kernel scores zero.
- Triton API level: Triton 3.x (tl.load/tl.store
    with masks, tl.dot,
  tl.constexpr, num_warps/num_stages launch kwargs,
     tl.exp2, tl.rsqrt, etc.).
- Common API pitfalls (Triton 3.6): there is NO tl.
    tanh and NO tl.math.tanh.
  For tanh use `from triton.language.extra import
    libdevice` then
  `libdevice.tanh(x)`, or the identity tanh(x) = 2*
    tl.sigmoid(2*x) - 1.
  erf exists as tl.math.erf (and libdevice.erf). tl
    .exp / tl.log / tl.sqrt /
  tl.rsqrt / tl.exp2 / tl.log2 / tl.sigmoid exist
    directly under tl.
  Allowed imports also include `from triton.
    language.extra import libdevice`.
```

APPENDIX C
REPRESENTATIVE AGENT OUTPUT

Verbatim model response for the RoPE candidate above (the generated kernel, $4.29\times$). The model also emits a one-line confidence record consumed by the harness.

````
```python
import torch
import triton
import triton.language as tl

@triton.jit
def rope_kernel(
    x_ptr, cos_ptr, sin_ptr, out_ptr,
    stride_x_b, stride_x_h, stride_x_s, stride_x_d,
    stride_cos_s, stride_cos_d,
    stride_sin_s, stride_sin_d,
    stride_out_b, stride_out_h, stride_out_s,
    stride_out_d,
    B: tl.constexpr, H: tl.constexpr, S: tl.
    constexpr, D: tl.constexpr,
    HALF_D: tl.constexpr,
    ROWS_PER_PROG: tl.constexpr,
):
    pid = tl.program_id(0)
    start_row = pid * ROWS_PER_PROG
````
````

```
    offs_half = tl.arange(0, HALF_D)

    for i in range(ROWS_PER_PROG):
        row = start_row + i
        # Decompose linear row index into (b, h, s)
        b = row // (H * S)
        rem = row % (H * S)
        h = rem // S
        s = rem % S

        x_base = x_ptr + b * stride_x_b + h *
    stride_x_h + s * stride_x_s
        out_base = out_ptr + b * stride_out_b + h *
     stride_out_h + s * stride_out_s

        # Load x halves (contiguous along last
    dimension)
        x1 = tl.load(x_base + offs_half,
    eviction_policy='evict_first')
        x2 = tl.load(x_base + (offs_half + HALF_D),
     eviction_policy='evict_first')

        # Load cos and sin for this sequence
    position -- keep in L2 for reuse across rows
        cos_base = cos_ptr + s * stride_cos_s
        sin_base = sin_ptr + s * stride_sin_s
        c = tl.load(cos_base + offs_half,
    eviction_policy='evict_last')
        s_val = tl.load(sin_base + offs_half,
    eviction_policy='evict_last')

        # Compute rotated halves
        out1 = x1 * c - x2 * s_val
        out2 = x1 * s_val + x2 * c

        # Store output
        tl.store(out_base + offs_half, out1)
        tl.store(out_base + offs_half + HALF_D,
    out2)


def triton_run(x: torch.Tensor, cos: torch.Tensor,
    sin: torch.Tensor) -> torch.Tensor:
    assert x.dtype == torch.float16
    assert cos.dtype == torch.float16 and sin.dtype
     == torch.float16
    B, H, S, D = x.shape
    assert D == 128
    assert cos.shape == (S, D // 2) and sin.shape
    == (S, D // 2)

    out = torch.empty_like(x)

    HALF_D = D // 2       # 64
    ROWS_PER_PROG = 16
    total_rows = B * H * S
    assert total_rows % ROWS_PER_PROG == 0, "
    total_rows must be divisible by ROWS_PER_PROG"

    grid = (total_rows // ROWS_PER_PROG,)

    rope_kernel[grid](
        x, cos, sin, out,
        x.stride(0), x.stride(1), x.stride(2), x.
    stride(3),
        cos.stride(0), cos.stride(1),
        sin.stride(0), sin.stride(1),
        out.stride(0), out.stride(1), out.stride(2)
    , out.stride(3),
        B, H, S, D,
        HALF_D,
        ROWS_PER_PROG,
        num_warps=4,
        num_stages=2,
    )
    return out
```
CONF: {"confidence": 90, "notes": "Changed cos/sin
    eviction to evict_last for better L2 reuse and
    increased ROWS_PER_PROG to 16 to reduce grid
    overhead."}
```

The best Engine B kernel (`int4_gemm`, 1.09×) is included in the repository at `runs/agent_int4_gemm/best.py`.

## APPENDIX D
## CHANGES MADE TO THE AGENT

We modified the harness and the agent contract during the project. (1) *Parameter-grid autotuning*: candidates now expose each tunable launch parameter as an LLM-chosen grid swept by `triton.autotune`, and the evaluator scans the autotuner to record the winning configuration in the log so later rounds narrow the grid. (2) *Transcript logging*: every LLM exchange (system prompt, input prompt, raw response, and the model's reasoning) is saved per candidate—these files are the source of this appendix. (3) *Model override*: a `--model` flag selects DEEPSEEK-V4-PRO vs `-flash` per run, recorded in the run metadata. (4) *Multi-process safety*: a spawn stagger and an optional startup jitter prevent simultaneous CUDA-context initialization from many evaluator subprocesses. (5) *Agent contract* (`HARNESS.md`, Engine B): the agent (CLAUDE CODE, model CLAUDE OPUS, reasoning effort "extra high") writes a kernel, calls the evaluator (`eval_one`) with a strategy label, and hands off state through a log-derived best kernel rather than conversation memory. The complete `HARNESS.md` and configuration files are in the repository.
```